

1. Enquadramento & Objetivos Pedagógicos

Atualmente, o desenvolvimento de soluções na área de Sistemas Computacionais Embebidos e Internet das Coisas (IoT) exige a integração contínua entre dispositivos na periferia da rede (Edge Computing) e plataformas centrais de processamento em nuvem (Cloud Computing). Embora os hipervisores tipo 1 (como o Proxmox VE) constituam a base de infraestruturas locais e de nevoeiro (Fog Computing), a ligação a plataformas como o Microsoft Azure permite expandir a monitorização, ingestão massiva de dados e gestão remota de dispositivos à escala global.

Para contornar a ausência de hardware físico dedicado (como microcontroladores ESP32, Raspberry Pi ou módulos sensores I2C/SPI), este guião implementa uma simulação de elevada fidelidade baseada em dados reais de sensores urbanos da plataforma Porto Digital / FIWARE. Os dados de qualidade do ar, meteorologia e ruído são recolhidos e publicados via MQTT, permitindo aos alunos testar arquiteturas completas de dados do nó de campo até à nuvem.

2. Descrição Detalhada das Máquinas no Proxmox VE

Para simular o ambiente industrial/urbano, o laboratório é estruturado sobre o hipervisor Proxmox VE. A arquitetura divide-se entre um Gateway Edge (VM KVM) e dois Containers LXC que desempenham o papel de Nó Embebido de Campo e Nuvem Local de Backup.

Tabela 2.1 — Especificação de Instâncias Virtuais no Proxmox VE

Instância / ID		Tipo & SO		vCPU / RAM / Disco		Rede & IP		Função no Laboratório
VM (vm-edge-gateway)	100	VM	KVM	2 2048 MB 20 GB Storage	vCPU RAM	Bridge: vmbr0 IP: 192.168.1.50/24 GW: 192.168.1.1		Gateway Edge, Docker Engine, Docker Compose, k3s, Azure IoT Bridge.
LXC (ct-fiware-sensor)	200	LXC	Container	1 1024 MB 10 GB Storage	vCPU RAM	Bridge: vmbr0 IP: 192.168.1.51/24 Features: nesting=1, keyctl=1		Emulador de Nó Embebido de Campo (recolha FIWARE Porto Digital / MQTT).
LXC (ct-local-cloud)	300	LXC	Container	1 1024 MB 10 GB Storage	vCPU RAM	Bridge: vmbr0 IP: 192.168.1.52/24 Features: nesting=1		Novem Local / Servidor de Ingestão de Backup (Proxmox-)

Cloud).

REQUISITO	OBRIGATÓRIO:	ATIVAÇÃO	DE	NESTING	EM	LXC
1.	No Proxmox VE Web UI, aceda a Datacenter -> pve -> Create CT (ID 200).					
2.	Escolha o template Ubuntu 22.04 e desmarque a caixa 'Unprivileged container' (ou configure idmap).					
3.	Após a criação, aceda a LXC 200 -> Options -> Features -> Edit -> Ativar 'Nesting' (nesting=1) e 'Keyctl' (keyctl=1).					
4.	Sem estas duas opções ativas, o motor de containers Docker e o driver Overlay2 não conseguirão criar namespaces dentro do container LXC.					

3. Guia do Iniciante para Microsoft Azure: Navegação, Rede e Poupança de Custos

Para alunos sem experiência prévia no Microsoft Azure, esta secção fornece um guia passo a passo de navegação no portal, criação de recursos de rede e serviços IoT, garantindo que o laboratório seja executado a ****custo zero (\$0)****.

a) Estratégia de Poupança de Custos para Alunos (\$0 / Custo Zero)

A Microsoft disponibiliza o programa Azure for Students (\$100 de crédito gratuito sem necessidade de cartão de crédito) ou a Conta Gratuita do Azure. Para evitar consumos acidentais de crédito ou cobranças:

- **Azure IoT Hub Free Tier (F1):** Escolha SEMPRE o escalão de preço F1 (Free). Permite o processamento de até 8.000 mensagens por dia gratuitamente (1 IoT Hub Grátis por subscrição).
- **Evitar Máquinas Virtuais Desnecessárias na Nuvem:** Todo o processamento pesado e simulação é feito nas instâncias locais do Proxmox VE. Na nuvem Azure utiliza-se apenas o serviço PaaS (IoT Hub). Caso deseje criar uma VM de teste de rede no Azure, escolha o tamanho Standard_B1ls (1 vCPU, 0.5 GB RAM) e desligue-a após a aula.
- **Eliminação do Resource Group no Final do Laboratório:** A forma mais eficaz de garantir custo zero é apagar todo o Grupo de Recursos no final do trabalho com um único comando.

b) Navegação Passo a Passo no Portal do Azure (<https://portal.azure.com>)**PASSO 1: Criar o Grupo de Recursos (Resource Group)**

1. Aceda a <https://portal.azure.com> e inicie sessão com a conta de estudante.
2. Na barra de pesquisa superior, pesquise por 'Resource groups' e clique em 'Create'.
3. Selecione a sua Subscrição (ex: Azure for Students).
4. Nome do Resource Group: rg-sistemas-embebidos-lab
5. Região: (Europe) West Europe ou North Europe -> Clique em 'Review + create' -> 'Create'.

PASSO 2: Criar uma Rede Virtual (Virtual Network - VNet)

1. Pesquise por 'Virtual networks' na barra superior -> Clique em 'Create'.
2. Resource Group: selecione 'rg-sistemas-embebidos-lab'.
3. Name: vnet-iot-lab | Region: West Europe.
4. IP Addresses: Espaço de endereçamento IPv4: 10.0.0.0/16.
5. Subnet Name: snet-devices (10.0.1.0/24) -> Clique em 'Review + create' -> 'Create'.

PASSO 3: Criar o Azure IoT Hub (F1 Free Tier)

1. Pesquise por 'IoT Hub' na barra superior -> Clique em 'Create'.
2. Resource Group: 'rg-sistemas-embebidos-lab'.
3. IoT Hub Name: iothub-embebidos-pve-XXXXX (substitua XXXXX pelas suas iniciais para ser único).
4. Separador 'Management' / 'Pricing and scale tier': Selecione 'F1: Free tier'.
5. Clique em 'Review + create' -> 'Create' (o aprovisionamento demora ~2 minutos).

PASSO 4: Registo de Dispositivo e Obtenção da Connection String

1. Aceda ao seu IoT Hub -> No menu esquerdo, em 'Device management', clique em 'Devices'.
2. Clique em '+ Add Device' -> Device ID: 'nos-embebidos-porto-01'.
3. Mantenha 'Symmetric key' e guarde -> Clique no dispositivo criado 'nos-embebidos-porto-01'.
4. Copie o campo 'Primary Connection String' (necessário para a ponte de telemetria).

COMANDO DE LIMPEZA OBRIGATÓRIO NO FINAL DA AULA

Para eliminar TODOS os recursos do Azure criados na aula e evitar qualquer custo residual, abra o Azure Cloud Shell (ícone >_ no topo do portal) e execute:

```
az group delete --name rg-sistemas-embebidos-lab --yes --no-wait
```

4. Desenrolar do Trabalho Laboratorial

a) AULA 1: Simulação de Dispositivos & Ponte Telegráfica MQTT para Azure / Proxmox Cloud

Instalação do Simulador de Campo no LXC 200:

```
# Executar no LXC 200 (ct-fiware-sensor)
apt update && apt install -y python3 python3-pip python3-venv mosquitto mosquitto-
clients
systemctl enable --now mosquitto

# Criar diretoria do projeto
mkdir -p /opt/fiware-sim && cd /opt/fiware-sim
python3 -m venv env && source env/bin/activate
pip install requests paho-mqtt python-dateutil azure-iot-device

# Descarregar o script de recolha e simulação de sensores do Porto Digital
wget https://raw.githubusercontent.com/jcboliveira/fiware\_mqtt/main/fiware\_mqtt.py

# Testar a recolha de dados reais da estação 'Paranhos' e publicação no Mosquitto
local
python3 fiware_mqtt.py --mqtt-host 127.0.0.1 --stations 'Paranhos,Ramalde'
```

Envio de Telemetria para a Nuvem (OPÇÃO A: Azure IoT Hub | OPÇÃO B: Proxmox Cloud):

Os alunos com acesso ao Azure utilizam o script `azure\_bridge.py`. Para alunos sem conta Azure ou para executar 100% offline em Proxmox, utiliza-se a OPÇÃO B (`local\_cloud\_bridge.py`) enviando os dados para a instância LXC 300 (`ct-local-cloud`).

```
# OPÇÃO A: Ponte para Azure IoT Hub (azure_bridge.py)
# Criar o ficheiro azure_bridge.py na VM 100 / LXC 200:
cat << 'EOF' > /opt/fiware-sim/azure_bridge.py
import json, time
import paho.mqtt.client as mqtt
from azure.iot.device import IoTHubDeviceClient

CONNECTION_STRING = 'HostName=iothub-embebidos-pve.azure-devices.net;DeviceId=nos-
embebidos-porto-01;SharedAccessKey=XXXXXXXX'
```

```
azure_client = IoTHubDeviceClient.create_from_connection_string(CONNECTION_STRING)
azure_client.connect()

def on_message(client, userdata, msg):
    try:
        payload = json.loads(msg.payload.decode())
        telemetry_data = {'topic': msg.topic, 'value': payload, 'timestamp':
time.time()}
        azure_client.send_message(json.dumps(telemetry_data))
        print(f'[AZURE IOT HUB] Telemetria enviada: {msg.topic}')
    except Exception as e:
        print(f'Erro ao reencaminhar: {e}')

local_mqtt = mqtt.Client('azure_bridge')
local_mqtt.on_message = on_message
local_mqtt.connect('127.0.0.1', 1883)
local_mqtt.subscribe('fiware/#')
local_mqtt.loop_forever()
EOF

python3 /opt/fiware-sim/azure_bridge.py
```

b) AULA 2: Stack Edge Local, Grafana Cloud e Gestão Remota (Device Twins)

Implantação da Stack Edge (Docker Compose) e Grafana Cloud:

A segunda aula implanta o ecossistema completo de observabilidade no Gateway Edge (VM 100), conectando a visualização local ao Grafana Cloud.

```
# Ficheiro docker-compose.yml em /opt/mqtt-fiware-grafana na VM 100
version: '3.8'
services:
  mosquitto:
    image: eclipse-mosquitto:2
    container_name: mosquitto_edge
    ports: ['1883:1883', '9001:9001']
    volumes: ['./mosquitto.conf:/mosquitto/config/mosquitto.conf']

  cratedb:
```

```
image: crate:5.4
container_name: cratedb_edge
ports: ['4200:4200', '5432:5432']
environment: ['CRATE_HEAP_SIZE=512m']

grafana:
image: grafana/grafana:10.0.0
container_name: grafana_edge
ports: ['3000:3000']
environment: ['GF_SECURITY_ADMIN_PASSWORD=admin']

alloy:
image: grafana/alloy:v1.0.0
container_name: grafana_alloy_cloud
volumes: ['./config.alloy:/etc/alloy/config.alloy']
ports: ['12345:12345']
```

Exercício Prático com Grafana Cloud (Exportação de Métricas):

```
# Configuração do Grafana Alloy (config.alloy) para envio de métricas para Grafana
Cloud
prometheus.remote_write 'grafana_cloud' {
  endpoint {
    url = 'https://prometheus-prod-01-eu-west-0.grafana.net/api/v1/push'
    basic_auth {
      username = 'SUA_USER_ID_GRAFANA_CLOUD'
      password = 'SEU_API_KEY_GRAFANA_CLOUD'
    }
  }
}
```

Gestão Remota e Reconciliação com Azure Device Twins:

Os alunos testam a alteração dinâmica da taxa de amostragem do nó simulado através do envio de propriedades desejadas (*Desired Properties*) a partir do Azure IoT Hub.

```
# Ficheiro device_twin_listener.py na VM 100
import time, json
from azure.iot.device import IoTHubDeviceClient
```

```
CONNECTION_STRING = 'HostName=iothub-embebidos-pve.azure-devices.net;DeviceId=nos-embebidos-porto-01;SharedAccessKey=XXXXXXX'
client = IoTHubDeviceClient.create_from_connection_string(CONNECTION_STRING)
client.connect()

def twin_patch_handler(patch):
    print(f'[AZURE DEVICE TWIN] Alteração de configuração recebida: {patch}')
    if 'sample_rate' in patch:
        new_rate = patch['sample_rate']
        print(f'-> AUSTANDO TAXA DE AMOSTRAGEM DO SENSOR PARA: {new_rate}s')
        client.patch_twin_reported_properties({'sample_rate': new_rate, 'status': 'APPLIED'})

client.on_twin_desired_properties_patch_received = twin_patch_handler
print('A aguardar atualizações do Device Twin no Azure Portal...')
while True:
    time.sleep(1)
```

5. Todos os Ficheiros, Scripts e Códigos Completos do Laboratório

a) Ficheiro `mosquitto.conf`

```
listener 1883
allow_anonymous true
persistence true
persistence_location /mosquitto/data/
log_dest stdout
```

b) Script de Nuvem Local de Backup (`local\_cloud\_bridge.py`) — Alternativa sem Azure

```
# Executa no LXC 300 (ct-local-cloud)
import paho.mqtt.client as mqtt, json, time

def on_msg(client, userdata, msg):
    print(f'[PROXMOX LOCAL CLOUD - INGESTÃO] Tópico: {msg.topic} | Dados:
```

```
{msg.payload.decode()[:60]}...')

cli = mqtt.Client('proxmox_cloud_ingest')
cli.on_message = on_msg
cli.connect('192.168.1.51', 1883)
cli.subscribe('fiware/#')
print('Servidor Cloud Local (Proxmox LXC 300) em escuta...')
cli.loop_forever()
```

6. Avaliação e Questões de Reflexão Crítica

Os alunos devem responder às seguintes questões no relatório técnico final:

- **Questão 1 (Fidelidade da Simulação vs. Hardware Físico Embebido):** Analise em que medida a simulação via dados reais do FIWARE/Porto Digital reflete os desafios de um sistema embebido físico. Quais as limitações desta abordagem no que concerne ao consumo de energia (Sleep Modes), restrições de memória SRAM e latência de rádio (LoRaWAN / NB-IoT)?
- **Questão 2 (Análise do Modelo de Custos e Segurança Cloud vs. Edge):** Compare a utilização do Azure IoT Hub (F1 Free Tier) em relação à Nuvem Local em Proxmox VE. Sob o ponto de vista da segurança, quais as vantagens da utilização de SAS Tokens e Certificados X.509 no Azure em relação ao MQTT não cifrado em redes de campo?
- **Questão 3 (Reconciliação de Estado com Device Twins):** Explique o conceito de 'Desired Properties' e 'Reported Properties' no mecanismo de Device Twins do Azure IoT Hub. Como é que este padrão garante a consistência do estado de um nó embebido que sofra interrupções temporárias de conectividade?

7. Recursos e Referências

- Repositório FIWARE/MQTT Porto Digital: https://github.com/jcboliveira/fiware_mqtt
- Repositório Stack MQTT/FIWARE/Grafana: <https://github.com/jcboliveira/mqtt-fiware-grafana>
- Documentação Oficial Microsoft Azure IoT Hub: <https://learn.microsoft.com/azure/iot-hub/>
- Documentação Oficial Grafana Cloud & Alloy: <https://grafana.com/docs/alloy/latest/>